Cahier de charge

Cahier des Charges Fonctionnel et Technique

Projet : Plateforme Intelligente de Génération et Gestion des Tests Logiciels

Réaliser par : Taha Khelifi & Mohamed hassouna

Sommaire

Introduction

Dans les environnements de développement Agile, et plus particulièrement Scrum, la qualité logicielle constitue un enjeu central pour la réussite des projets informatiques. Les cycles de développement courts et les livraisons fréquentes imposent aux équipes une forte réactivité, ce qui rend les activités de test souvent complexes, chronophages et difficiles à suivre. La génération manuelle des cahiers de tests, l'exécution des tests et la production des rapports de qualité manquent fréquemment de centralisation, de traçabilité et d'automatisation.

Les tests logiciels, qu'ils soient unitaires, automatisés ou manuels, jouent pourtant un rôle essentiel dans la validation des fonctionnalités et la détection des anomalies. L'intégration de l'intelligence artificielle dans ces processus offre une opportunité importante pour automatiser la génération des tests, analyser les résultats et améliorer la qualité globale des applications.

Dans ce contexte, ce projet vise à concevoir et développer une Plateforme Intelligente de Génération et Gestion des Tests Logiciels (Scrum – Tests Automatisés – IA – QA). Cette plateforme permettra de centraliser la gestion des tests dans un cadre Scrum, de faciliter la collaboration entre les différents acteurs du projet et d'automatiser la génération des cahiers de tests, des rapports QA et des recommandations d'amélioration, en s'appuyant sur des technologies modernes telles que Next.js, FastAPI et PostgreSQL.

Objectif du projet

Objectif général :

L'objectif général de ce projet est de concevoir et développer une Plateforme Intelligente de Génération et Gestion des Tests Logiciels permettant d'automatiser les activités de tests (tests unitaires, tests automatisés et manuels) dans un environnement Agile Scrum, afin d'améliorer la qualité logicielle, d'optimiser le temps de test et de faciliter le suivi de la qualité tout au long du cycle de développement.

Objectif spécifique

Les objectifs spécifiques du projet sont les suivants :

- Centraliser la gestion des projets Scrum, des modules, des epics, des user stories et des sprints.
- Générer automatiquement un cahier de tests structuré à partir des user stories.
- Automatiser la génération des tests unitaires pour les développeurs.
- Générer et gérer des tests automatisés et manuels pour les testeurs QA.
- Assurer le suivi et la traçabilité des résultats des tests (succès, échecs, logs).

- Générer des rapports QA détaillés à la fin de chaque sprint.
- Proposer des recommandations intelligentes d'amélioration basées sur l'analyse des résultats des tests.
- Faciliter la collaboration et la communication entre les différents acteurs du projet.
- Mettre en place un système de notifications automatiques pour informer les utilisateurs des événements importants.

Périmètre du projet

Le périmètre de ce projet couvre la conception, le développement et le déploiement d'une plateforme web intelligente dédiée à la génération, la gestion et le suivi des tests logiciels dans un environnement Agile Scrum.

Inclus dans le périmètre du projet

Le projet comprend les éléments suivants :

- La mise en place d'une application web accessible via navigateur.
- La gestion des projets Scrum, incluant modules, epics, user stories et sprints.
- La génération automatique d'un cahier de tests structuré à partir des user stories.
- La génération automatique des tests unitaires pour les développeurs.
- La génération et la gestion des tests automatisés et manuels pour les testeurs QA.
- Le suivi de l'exécution des tests et la gestion des résultats, statuts et logs.
- La génération de rapports QA en fin de chaque sprint.
- La génération de recommandations intelligentes d'amélioration basées sur les résultats des tests.
- La gestion des rôles, permissions et accès des différents acteurs du système.
- L'envoi de notifications automatiques liées aux événements clés (tests, sprints, rapports).
- L'utilisation des technologies : Next.js, FastAPI et PostgreSQL.

Hors périmètre du projet

Les éléments suivants ne sont pas inclus dans le périmètre :

- Le développement ou la maintenance d'applications clientes externes.
- Les tests matériels ou embarqués.
- La correction automatique du code source des applications testées.

- L'hébergement et l'infrastructure cloud avancée (hors déploiement de base).
- L'intégration avec des outils tiers complexes non prévus initialement.

Acteurs du système

Chaque acteur du système dispose d'un rôle bien défini lui permettant d'interagir avec la plateforme selon ses responsabilités dans le cadre du processus Agile Scrum et de l'assurance qualité logicielle.

Acteur	Description
Super Administrateur	Le Super Administrateur est responsable de la gestion globale de la plateforme. Il configure les paramètres système, administre les comptes utilisateurs, définit les rôles et permissions, et assure la sécurité, la disponibilité et la maintenance technique et fonctionnelle de la plateforme.
Product Owner (Admin)	Le Product Owner est chargé de définir les besoins fonctionnels du produit sous forme de modules et d'epics. Il priorise les fonctionnalités, valide les livrables, suit l'avancement du projet et s'assure que le produit répond aux exigences métier et aux objectifs de qualité.
Scrum Master	Le Scrum Master organise et supervise le processus Scrum. Il décompose les epics en user stories, planifie les sprints, coordonne les équipes et veille au respect des bonnes pratiques Agile tout au long du cycle de développement.
Développeur	Le Développeur implémente les fonctionnalités décrites dans les user stories. Il développe le code applicatif, génère et exécute les tests unitaires, corrige les anomalies détectées et contribue à l'amélioration continue de la qualité du logiciel.
Testeur (QA)	Le Testeur (QA) est responsable de la validation du logiciel. Il conçoit, génère et exécute des tests automatiques ou manuels, analyse les résultats, documente les défauts et participe à l'évaluation de la qualité en fin de sprint.
Système d'Intelligence	Le Système d'IA agit comme un acteur automatisé. Il analyse les user stories et les données du projet afin de générer automatiquement les cahiers

Artificielle (IA)	de tests, les tests unitaires et automatisés, les rapports QA de fin de sprint ainsi que des recommandations d'amélioration.

Description des interfaces

L'application propose des interfaces web ergonomiques, sécurisées et adaptées aux rôles des différents acteurs du système.

Les interfaces sont développées avec Next.js pour assurer une navigation fluide, une bonne performance et une expérience utilisateur intuitive.

Chaque interface est accessible selon les droits et responsabilités de l'acteur concerné.

Interface Super Administrateur

Référence : INT-SA-01

L'interface du Super Administrateur permet la gestion globale et la supervision complète de la plateforme.

Fonctionnalités principales :

- Gestion des comptes utilisateurs (création, modification, suppression).
- Attribution et gestion des rôles et permissions (Product Owner, Scrum Master, Développeur, Testeur).
- Accès à un tableau de bord global présentant :
  - Nombre de projets actifs.
  - Nombre de sprints réalisés.
  - Activité globale des utilisateurs.
- Supervision des données système et des historiques.
- Consultation des logs techniques et fonctionnels.
- Paramétrage global de la plateforme.

Interface Product Owner (Admin)

Référence : INT-PO-02

L'interface du Product Owner est dédiée à la définition et au pilotage des besoins métier.

Fonctionnalités principales :

- Création et gestion des projets.
- Définition des modules et des epics.

- Priorisation des besoins fonctionnels.
- Consultation de l'avancement des sprints.
- Validation des livrables et des résultats de tests.
- Accès aux rapports QA globaux.

Interface Scrum Master

Référence : INT-SM-03

L'interface du Scrum Master permet l'organisation et le suivi du processus Scrum.

Fonctionnalités principales :

- Décomposition des epics en user stories.
- Planification et gestion des sprints.
- Affectation des user stories aux développeurs.
- Suivi de l'état des tâches et des tests associés.
- Consultation des indicateurs de qualité par sprint.
- Coordination entre les acteurs du projet

Interface Développeur

Référence : INT-DEV-04

L'interface du Développeur est orientée vers l'implémentation et la qualité du code.

Fonctionnalités principales :

- Consultation des user stories assignées.
- Accès aux spécifications fonctionnelles.
- Génération automatique des tests unitaires à partir des user stories.
- Consultation des résultats des tests unitaires.
- Correction des anomalies détectées.
- Suivi de l'état d'avancement du développement.

Interface Testeur (QA)

Référence : INT-QA-05

L'interface du Testeur (QA) est dédiée à la validation fonctionnelle et technique de l'application.

Fonctionnalités principales :

- Création et gestion des scénarios de test.

- Génération et exécution des tests automatiques.
- Gestion des tests manuels.
- Consultation des résultats des tests (succès / échec).
- Ajout de logs d'erreurs, captures et commentaires.
- Suivi de la qualité logicielle par sprint.

Interface Rapports et Recommandations QA

Référence : INT-RPT-06

Cette interface permet la visualisation et l'analyse des résultats de qualité.

Fonctionnalités principales :

- Génération automatique des rapports QA en fin de sprint.
- Visualisation des indicateurs de qualité (taux de réussite, défauts).
- Historique des rapports QA.
- Affichage des recommandations d'amélioration générées par l'IA.
- Aide à la prise de décision pour les sprints suivants.

Interface Système d'Intelligence Artificielle

Référence : INT-IA-07

Le système d'IA agit comme un composant applicatif automatisé.

Fonctionnalités principales :

- Analyse des user stories et des modules.
- Génération automatique :
  - Du cahier de tests,
  - Des tests unitaires,
  - Des tests automatisés.
- Génération des rapports QA.
- Proposition de recommandations techniques et fonctionnelles.
- Support à l'amélioration continue de la qualité logicielle.

Rôles, permissions et accès

Cette section définit les différents rôles du système ainsi que leurs permissions et niveaux d'accès à la plateforme.

Chaque rôle est associé à une ou plusieurs interfaces identifiées par une référence unique, garantissant une gestion sécurisée et cohérente des accès.

Tableau des rôles, permissions et interfaces associées

Rôle	Interface associée (Référence)	Permissions principales	Niveau d'accès
Super Administrateur	Interface Super Administrateur (INT-SA-01)	- Gestion globale de la plateforme - Gestion des comptes utilisateurs - Attribution des rôles et permissions - Accès aux logs et paramètres système - Supervision des projets et données	Accès total
Product Owner (Admin)	Interface Product Owner (INT-PO-02)	- Création et gestion des projets - Définition des modules et epics - Priorisation des besoins - Validation des livrables - Consultation des rapports QA	Accès élevé
Scrum Master	Interface Scrum Master (INT-SM-03)	- Décomposition des epics en user stories - Planification et gestion des sprints - Affectation des tâches - Suivi de l'avancement et de la qualité par sprint	Accès fonctionnel
Développeur	Interface Développeur (INT-DEV-04)	- Consultation des user stories assignées - Développement des fonctionnalités - Génération et exécution des tests unitaires - Correction des anomalies	Accès limité

Testeur (QA)	Interface Testeur (INT-QA-05)	(INT-QA-05) - Création et gestion des scénarios de test - Exécution des tests automatiques et manuels - Consultation des résultats - Ajout de logs, captures et commentaires	Accès limité
Utilisateur QA / Décideur	Interface Rapports & Recommandations (INT-RPT-06)	- Consultation des rapports QA - Analyse des indicateurs de qualité - Consultation des recommandations IA	Accès en lecture
Système d'Intelligence Artificielle	Interface IA (INT-IA-07)	- Génération automatique des tests - Génération des rapports QA - Analyse des résultats - Proposition de recommandations	Accès interne (automatisé)

Politique de contrôle d'accès

- L'accès aux interfaces est contrôlé par un mécanisme d'authentification et d'autorisation.
- Chaque utilisateur se voit attribuer un rôle unique ou multiple, défini par le Super Administrateur.
- Les permissions sont accordées selon le principe du moindre privilège, garantissant la sécurité et la confidentialité des données.
- Le Système d'Intelligence Artificielle fonctionne comme un acteur interne sans accès direct à l'interface utilisateur.

Traçabilité et sécurité

- Toutes les actions des utilisateurs sont journalisées (logs).
- Les modifications critiques (création, suppression, validation) sont historisées.
- Les accès et actions sont traçables par rôle et par interface.

NB : « Utilisateur QA / Décideur » n'est pas un acteur métier principal, mais un rôle de consultation (vue lecture) dérivé des autres acteurs.

Explication des niveaux d’accès

Accès total

Description :

Ce niveau d’accès offre un contrôle complet sur l’ensemble de la plateforme, aussi bien sur les aspects fonctionnels que techniques.

Caractéristiques :

- Accès à toutes les interfaces et fonctionnalités.
- Gestion des utilisateurs, rôles et permissions.
- Consultation et modification des paramètres système.
- Accès aux logs, historiques et données globales.

Accès élevé

Description :

Ce niveau d’accès permet de piloter le produit et de prendre des décisions stratégiques sans intervenir sur la configuration technique du système.

Caractéristiques :

- Création et gestion des projets.
- Définition et priorisation des besoins (modules, epics).
- Validation des livrables.
- Consultation des rapports QA et indicateurs globaux.
- Aucune modification des paramètres système critiques.

Accès fonctionnel

Description :

Ce niveau d’accès est dédié à la gestion opérationnelle du processus Agile Scrum.

Caractéristiques :

- Gestion des user stories et des sprints.
- Affectation des tâches aux développeurs.
- Suivi de l’avancement et de la qualité par sprint.
- Accès aux indicateurs liés au déroulement des sprints.
- Pas d’accès à la configuration globale du système.

Accès limité

Description :

Ce niveau d'accès permet l'exécution de tâches spécifiques liées au développement et aux tests, sans accès aux données stratégiques ou globales.

Caractéristiques :

- Accès uniquement aux éléments assignés.
- Génération et exécution des tests.
- Consultation des résultats et ajout de commentaires.
- Modification restreinte aux données opérationnelles.

Accès en lecture seule

Description :

Ce niveau d'accès est réservé aux utilisateurs ayant un rôle d'analyse ou de prise de décision, sans interaction directe sur les données.

Caractéristiques :

- Consultation des rapports QA.
- Visualisation des indicateurs de qualité.
- Accès aux recommandations générées par l'IA.
- Aucune création, modification ou suppression de données.

Accès interne (automatisé)

Description :

Ce niveau d'accès est réservé aux composants internes du système et ne concerne pas les utilisateurs humains.

Caractéristiques :

- Accès contrôlé aux données nécessaires à l'analyse.
- Génération automatique des tests, rapports et recommandations.
- Aucune interaction via interface utilisateur.
- Fonctionnement sécurisé et autonome.

Exigences non fonctionnelles

Les exigences non fonctionnelles définissent les critères de qualité, de performance, de sécurité et de fiabilité que doit respecter la plateforme afin d'assurer une utilisation efficace, sécurisée et évolutive dans un environnement Agile Scrum.

Elles garantissent la satisfaction des utilisateurs, la robustesse du système et sa capacité à évoluer avec les besoins du projet.

Performance

La plateforme doit offrir des performances optimales afin de ne pas impacter le déroulement des sprints et des activités de test.

- Temps de réponse des fonctionnalités critiques (génération de tests, consultation des rapports, navigation) inférieur à 2 secondes.
- Génération automatique du cahier de tests, des tests unitaires et des tests automatisés optimisée pour un traitement rapide.
- Gestion fluide de plusieurs projets, sprints et tests exécutés simultanément.
- Consultation simultanée des rapports QA sans dégradation des performances.

Sécurité

La sécurité est un élément fondamental afin de protéger les données projets, les résultats de tests et les informations des utilisateurs.

- Authentification sécurisée des utilisateurs (login/mot de passe, tokens).
- Gestion des accès basée sur les rôles (RBAC – Role Based Access Control).
- Permissions strictes selon le rôle (Super Administrateur, Product Owner, Scrum Master, Développeur, Testeur).
- Chiffrement des données sensibles stockées et échangées.
- Journalisation et traçabilité des actions critiques :
  - Création et modification des projets,
  - Génération des tests,
  - Validation des livrables,
  - Génération des rapports QA.

Scalabilité

La plateforme doit être conçue pour accompagner l'évolution des projets et des équipes.

- Support de plusieurs projets Scrum et équipes simultanément.
- Architecture évolutive permettant l'ajout futur de nouvelles fonctionnalités QA sans refonte majeure.
- Capacité à gérer un volume croissant :
  - Des user stories,
  - De scénarios de test,
  - De résultats et historiques de tests.
- Compatibilité avec l'intégration future d'outils externes (CI/CD, outils de test).

Disponibilité et fiabilité

La plateforme doit garantir une disponibilité élevée et la continuité du service.

- Mise en place de mécanismes de sauvegarde et de restauration des données.
- Historisation des données pour éviter toute perte d'information.
- Taux de disponibilité cible de 99 %, assurant un accès quasi permanent à la plateforme.
- Tolérance aux erreurs et gestion des exceptions afin de maintenir la stabilité du système.

Choix techniques et justification

Frontend

Next.js (React Framework)

Next.js est utilisé pour le développement de la partie frontend de l'application.

Justification :

- Permet la création d'interfaces utilisateur dynamiques, modernes et ergonomiques.
- Offre de très bonnes performances grâce au rendu côté serveur (SSR) et au rendu statique.
- Améliore l'expérience utilisateur grâce à une navigation fluide.
- Facilite la structuration des interfaces selon les rôles (Super Administrateur, PO, Scrum Master, Développeur, Testeur).
- Large écosystème et forte communauté.

Backend

FastAPI (Python)

FastAPI est utilisé pour le développement de la partie backend et des API REST.

Justification :

- Framework rapide, léger et performant, basé sur Python.
- Idéal pour la création d'API REST sécurisées.
- Excellente intégration avec les technologies d'Intelligence Artificielle.
- Documentation automatique des API (Swagger / OpenAPI).
- Architecture claire facilitant la maintenance et l'évolution du système.

Base de données

PostgreSQL

PostgreSQL est utilisé comme système de gestion de base de données relationnelle.

Justification :

- Base de données robuste, fiable et performante.
- Gestion efficace des données structurées (projets, user stories, tests, résultats).
- Support avancé des relations, contraintes et transactions.
- Haute scalabilité et sécurité des données.
- Compatible avec les architectures modernes et les API REST.

Intelligence Artificielle

Modèles de langage (LLM / IA)

Les technologies d'Intelligence Artificielle sont utilisées pour automatiser les processus QA.

Justification :

- Génération automatique du cahier de tests.
- Génération intelligente des tests unitaires et automatisés.
- Analyse des résultats de tests.
- Génération de rapports QA et de recommandations d'amélioration.
- Gain de temps significatif et amélioration de la qualité logicielle.

Outils de tests et DevOps

Outils de tests automatisés (Playwright / Selenium) & **Docker et CI/CD**

Justification :

- Automatisation des tests fonctionnels et end-to-end.

- Déploiement rapide et reproductible.
- Intégration continue facilitant la qualité à chaque sprint.
- Réduction des erreurs humaines et amélioration de la fiabilité.

Justification globale des choix technologiques

Les choix technologiques retenus permettent :

- Une rapidité de développement grâce à des frameworks modernes.
- Une architecture modulaire et maintenable.
- Une excellente performance côté client et serveur.
- Une sécurité renforcée via la gestion des rôles et permissions.
- Une évolutivité facilitant l'ajout de nouvelles fonctionnalités QA.
- Un coût maîtrisé grâce à l'utilisation de technologies open-source.
- Une intégration native de l'IA, essentielle pour les objectifs du projet.

Modules du système et estimation

Le système est structuré en plusieurs modules fonctionnels, chacun répondant à un besoin précis du processus Agile Scrum et de l'assurance qualité logicielle.

Cette modularité facilite le développement, la maintenance et l'évolution de la plateforme.

Tableau des modules et estimations

N°	Module	Description	Acteurs concernés	Estimation(S/J)
M1	Authentification & Sécurité	Gestion des comptes, authentification, autorisation RBAC, gestion des rôles et permissions	Super Administrateur, Tous les utilisateurs	1
M2	Gestion des projets Scrum	Création et gestion des projets, modules, epics, user stories et sprints	Product Owner, Scrum Master	2
M3	Génération du cahier de tests	Génération automatique d'un cahier de tests structuré à partir des user stories	Système IA, Testeur, PO	2

M4	Génération des tests unitaires	Analyse des user stories et génération automatique des tests unitaires	Développeur, Système IA	2
M5	Génération des tests automatisés	Génération de scripts de tests automatiques et gestion des tests manuels	Testeur, Système IA	2
M6	Validation des tests	Validation manuelle ou automatique des tests générés, approbation des résultats et décision Go / No-Go	Testeur QA, Product Owner, Décideur QA	1
M7	Exécution & suivi des tests	Lancement des tests, suivi des statuts, gestion des échecs et historiques	Testeur, Développeur	1
M8	Rapports QA	Génération des rapports qualité par sprint (KPI, taux de réussite, anomalies)	QA, PO, Décideur	1
M9	Recommandations IA	Analyse des résultats et génération de recommandations d'amélioration	Système IA	2
M10	Notifications & alertes	Notifications automatiques (fin de tests, échecs, fin de sprint)	Tous les acteurs	1
M11	Administration & supervision	Supervision globale de la plateforme, logs, statistiques, configuration	Super Administrateur	1

Justification de l'estimation

Les estimations ont été établies en tenant compte :

- De la complexité fonctionnelle de chaque module,

- De l’intégration de l’Intelligence Artificielle,
- De l’architecture choisie (Next.js, FastAPI, PostgreSQL),
- Du développement itératif selon la méthodologie Scrum.

Planning par versions

Le développement de la plateforme sera réalisé selon une approche itérative et incrémentale, conforme à la méthodologie Agile Scrum.
Chaque version apporte un ensemble de fonctionnalités cohérentes et validables, permettant une montée en charge progressive du système.

Version 0 – Initialisation et Conception (V0)

Objectifs :

- Poser les fondations techniques et fonctionnelles du projet
- Valider la vision globale du système

Fonctionnalités / Livrables :

- Analyse détaillée des besoins
- Rédaction et validation du cahier des charges
- Modélisation UML (diagrammes de cas d’utilisation, classes, séquences)
- Définition de l’architecture globale
- Mise en place de l’environnement technique :
    - Frontend : Next.js
    - Backend : FastAPI
    - Base de données : PostgreSQL
- Authentification de base
- Gestion initiale des rôles (Super Administrateur, Product Owner, Scrum Master, Développeur, Testeur QA)

Version 1 – MVP (Minimum Viable Product)

Première version fonctionnelle de la plateforme.

Fonctionnalités principales :

- Gestion des projets Scrum
- Création et gestion des modules et epics

- Décomposition des epics en user stories
- Planification et gestion des sprints
- Interfaces principales par acteur
- Génération basique du cahier de tests
- Génération initiale des tests unitaires
- Suivi simple de l'état des tests

Version 2 – Automatisation des Tests et Validation QA

Fonctionnalités principales :

- Génération automatique avancée des tests unitaires
- Génération des tests automatisés (scripts de test)
- Exécution des tests automatiques
- Module de validation des tests (acceptation / rejet)
- Gestion des tests manuels
- Historique détaillé des tests
- Notifications automatiques (échec de test, fin de sprint)

Version 3 – Intelligence Artificielle et Reporting Avancé

Fonctionnalités principales :

- Génération automatique des rapports QA par sprint
- Tableaux de bord qualité (taux de réussite, couverture des tests, anomalies)
- Analyse intelligente des résultats de tests
- Génération de recommandations d'amélioration basées sur l'IA
- Optimisation des performances et de l'expérience utilisateur (UI/UX)

Évolutions futures

La Plateforme Intelligente de Génération et Gestion des Tests Logiciels a été conçue de manière modulaire et évolutive, afin de permettre l'intégration de nouvelles fonctionnalités et l'adaptation aux besoins futurs des équipes Agile et QA.

Les évolutions envisagées concernent aussi bien les aspects fonctionnels, techniques que stratégiques.

Amélioration des capacités d'Intelligence Artificielle

Intégration de modèles IA plus avancés pour :

- L'analyse prédictive des défauts logiciels
- La détection automatique des zones à risque dans le code
- Génération de tests basée sur l'historique des anomalies
- Amélioration continue des recommandations grâce au machine learning

Extension des types de tests supportés

- Ajout de nouveaux types de tests :
  - Tests de performance
  - Tests de charge
  - Tests de sécurité
- Génération automatique de scénarios de tests non fonctionnels
- Intégration avec des outils spécialisés (ex. tests de sécurité automatisés)

Intégration avec des outils DevOps externes

- Connexion avec des plateformes CI/CD (GitHub Actions, GitLab CI, Jenkins)
- Déclenchement automatique des tests à chaque commit ou pull request
- Synchronisation avec des outils de gestion de projet (Jira, Trello, Azure DevOps)

Personnalisation avancée et multi-projets

- Gestion multi-organisations et multi-équipes
- Paramétrage avancé des workflows QA par projet
- Personnalisation des rapports QA selon les besoins métier

Amélioration de l'expérience utilisateur

- Optimisation continue de l'interface utilisateur (UI/UX)
- Ajout de tableaux de bord interactifs et personnalisables
- Support multi-langue pour une adoption internationale

Ouverture vers une version SaaS

- Déploiement de la plateforme sous forme de solution SaaS
- Gestion des abonnements et des quotas d'utilisation
- Scalabilité horizontale pour supporter un grand nombre d'utilisateurs

Livrables

Dans le cadre du développement de la Plateforme Intelligente de Génération et Gestion des Tests Logiciels (Scrum – Tests Automatisés – IA – QA), plusieurs livrables seront produits tout au long du cycle de vie du projet.

Ces livrables permettent d'assurer la traçabilité, la validation des étapes et la qualité globale du système développé.

Les livrables sont organisés par phase et par version du projet.

Livrables de la phase de conception (Version 0)

- Cahier des charges fonctionnel et technique validé
- Analyse des besoins et périmètre du projet
- Modélisation UML :
  - Diagrammes de cas d'utilisation
  - Diagramme de classes
  - Diagrammes de séquence
  - Diagrammes d'activité
- Architecture globale du système
- Choix techniques et justification
- Planning prévisionnel par versions

Livrables de la phase de développement (Version 1 – MVP)

- Application web fonctionnelle (frontend Next.js)
- API backend opérationnelle (FastAPI)
- Base de données PostgreSQL structurée
- Gestion des utilisateurs et des rôles
- Interfaces par acteur (Super Administrateur, PO, Scrum Master, Développeur, Testeur QA)
- Gestion des projets Scrum (modules, epics, user stories, sprints)
- Génération basique du cahier de tests
- Documentation technique initiale

Livrables de la phase d'automatisation QA (Version 2)

- Module de génération automatique des tests unitaires
- Module de génération des tests automatisés
- Module de validation des tests
- Exécution et suivi des tests
- Historique et traçabilité des tests
- Système de notifications
- Rapports intermédiaires de qualité

Livrables de la phase IA et reporting (Version 3)

- Rapports QA complets par sprint
- Tableaux de bord qualité
- Module de recommandations intelligentes
- Analyse IA des résultats de tests
- Optimisation des performances et de l'UX
- Documentation utilisateur finale

Livrables finaux du projet

- Plateforme complète et déployée
- Code source documenté
- Documentation technique et fonctionnelle
- Guide d'utilisation par acteur
- Rapport final du projet
- Présentation finale

Index

Ce glossaire présente les acronymes et termes techniques utilisés dans le présent cahier des charges, afin de faciliter la compréhension du document par l'ensemble des parties prenantes.

Agile : Méthodologie de gestion de projet basée sur des itérations courtes et une amélioration continue.

- API (Application Programming Interface) : Interface permettant la communication entre différentes applications.

- Backend : Partie serveur de l’application, responsable de la logique métier et du traitement des données.
- CI/CD (Continuous Integration / Continuous Deployment) : Ensemble de pratiques visant à automatiser l’intégration, les tests et le déploiement du code.
- Epic : Ensemble d’un user stories représentant une fonctionnalité majeure du produit.
- FastAPI : Framework Python moderne utilisé pour le développement d’API rapides et performantes.
- Frontend : Partie visible de l’application avec laquelle l’utilisateur interagit.
- IA (Intelligence Artificielle) : Ensemble de techniques permettant à un système de simuler des capacités humaines comme l’analyse et la génération automatique.
- J/S (Jour/Homme) : Unité d’estimation correspondant à une journée de travail .
- LLM (Large Language Model) : Modèle de langage utilisé pour la génération automatique de contenus (tests, rapports, recommandations).
- Next.js : Framework React permettant le développement d’applications web performantes et optimisées pour le SEO.
- PostgreSQL : Système de gestion de base de données relationnelle robuste et fiable.
- QA (Quality Assurance) : Ensemble des activités visant à garantir la qualité logicielle.
- RBAC (Role-Based Access Control) : Modèle de gestion des accès basé sur les rôles des utilisateurs.
- Scrum : Framework Agile structurant le développement autour de rôles, d’événements et de sprints.
- Sprint : Itération de durée fixe durant laquelle un incrément du produit est développé.
- Test automatisé : Test exécuté automatiquement à l’aide d’outils ou de scripts.
- Test unitaire : Test visant à vérifier le bon fonctionnement d’une unité de code.
- User Story : Description simple d’un besoin utilisateur exprimée du point de vue métier.

Conclusion

Le présent cahier des charges a permis de définir de manière claire et structurée les objectifs, le périmètre, les acteurs, les fonctionnalités, l'architecture technique ainsi que les livrables de la Plateforme Intelligente de Génération et Gestion des Tests Logiciels.

Ce projet s'inscrit dans une démarche d'amélioration continue de la qualité logicielle au sein des environnements Agile Scrum, en répondant aux problématiques liées à la complexité, au coût et au caractère répétitif des activités de test.

La solution proposée vise à automatiser la génération des cahiers de tests, des tests unitaires et des tests automatisés, tout en assurant un suivi qualité rigoureuse à chaque sprint. Grâce à l'intégration de technologies modernes telles que Next.js, FastAPI, PostgreSQL et des mécanismes d'intelligence artificielle, la plateforme offrira une architecture modulaire, performante, sécurisée et évolutive.

En facilitant la collaboration entre le Product Owner, le Scrum Master, les développeurs, les testeurs QA et les décideurs, le système contribuera à une meilleure visibilité sur l'état de la qualité logicielle, à une réduction des erreurs humaines et à une optimisation des délais de livraison.

La génération automatique de rapports QA et de recommandations intelligentes permettra également de soutenir la prise de décision et l'amélioration continue des produits développés.

Enfin, cette plateforme ouvre des perspectives d'évolution futures, telles que l'intégration avec des outils DevOps, l'enrichissement des capacités d'analyse IA, l'extension à d'autres types de projets logiciels ou encore l'automatisation avancée des pipelines de tests, garantissant ainsi la pérennité et la valeur ajoutée de la solution dans un contexte technologique en constante évolution.